

W13D05 — Indexing Dasar & Query Performance

ML Engineer Journey · Phase 4: Database & Observability · 5 Jun 2026

1. Analogi: Perpustakaan Tanpa Katalog

Bayangkan perpustakaan 500.000 buku tanpa katalog. Mencari satu buku berarti jalan dari rak pertama, baca satu per satu hingga ketemu. Itulah **Sequential Scan** — PostgreSQL membaca setiap baris dari awal tabel hingga akhir. Untuk 200.000 baris, ini berarti 199.980 baris dibuang hanya untuk mendapat 20 hasil.

Index adalah katalog itu. Struktur data terpisah yang menyimpan nilai kolom beserta pointer ke lokasi fisik baris (disebut **ctid**), diurutkan sehingga database bisa melompat langsung ke target tanpa membaca semua baris.

2. Struktur B-Tree (Default Index Postgres)

B-Tree = Balanced Tree. Setiap leaf node berada di kedalaman yang sama, sehingga worst-case lookup selalu $O(\log n)$ — tidak peduli data ada di mana. Untuk 200.000 baris, B-Tree hanya butuh ~17 langkah ($\log_2 200000 \approx 17$) dibanding Seq Scan yang membaca semua baris.

Setiap node internal menyimpan *separator key* untuk routing. Leaf node menyimpan nilai kolom beserta ctid (pointer ke heap row). Index dibaca dari atas ke bawah sampai menemukan nilai, lalu lompat ke baris fisik di tabel.

Perbandingan nyata dari eksperimen: Seq Scan 13.233 ms → Index Scan 0.275 ms (48× lebih cepat, 200k baris, user_id lookup)

3. Jenis-Jenis Index di PostgreSQL

Jenis Index	Cocok untuk	Tidak cocok untuk
B-Tree (default)	=, <, >, BETWEEN, LIKE 'abc%', ORDER BY	LIKE '%abc%', full-text search
Hash	= saja, exact match lebih cepat	Range query apapun
GIN	Array, JSONB, full-text (@@)	Numeric range
GiST	Geometri, range types, full-text	Simple equality
BRIN	Tabel besar terurut fisik (time-series)	Random-access lookup

Untuk kebutuhan ML API — filter by id, email, timestamp range, status — B-Tree sudah mencukupi 99% kasus.

4. Kapan Index Worth It — dan Kapan Tidak

Index bukan gratis. Setiap INSERT, UPDATE, DELETE memaksa PostgreSQL memperbarui semua index yang terdampak — B-Tree harus di-rebalance. Semakin banyak index, semakin berat operasi tulis.

Biaya index yang sering dilupakan:

- DML lebih lambat — setiap write → semua index ikut diupdate
- Disk space — index bisa 10–30% ukuran tabel (terukur: score index 4416 kB dari tabel 200k baris)
- Cache pressure — index memakan buffer pool, mengurangi ruang untuk data tabel
- Planner overhead — semakin banyak index, semakin banyak candidate plan yang dievaluasi

Kondisi	Rekomendasi	Alasan
Tabel < 10k baris	Tidak perlu	Data fit di cache, Seq Scan lebih cepat
Tabel besar + high cardinality	Buat index	Speedup signifikan, index sangat selektif
Tabel besar + low cardinality	Hati-hati	>20-30% rows returned → Seq Scan lebih murah
Kolom sering di WHERE/JOIN	Kandidat index	Planner aktif menggunakan untuk filter
Kolom boolean/status (3 nilai)	Partial index	Full index tidak selektif, partial lebih hemat

5. Syntax Penting

Basic index:

```
CREATE INDEX idx_predictions_user_id
  ON ml_predictions (user_id);

-- Konvensi nama: idx_{table}_{column}
```

Composite index (urutan kolom penting):

```
-- Equality filter duluan, sort column terakhir
CREATE INDEX idx_ml_pred_model_status_date
  ON ml_predictions (model_name, status, created_at DESC);
```

Partial index (subset data):

```
-- Hanya index baris dengan status pending
CREATE INDEX idx_predictions_pending
  ON ml_predictions (created_at)
  WHERE status = 'pending';

-- Hasilnya: 352 kB vs 4416 kB untuk full index
```

Ukur ukuran index:

```
SELECT
  indexrelname AS index_name,
  pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
WHERE relname = 'ml_predictions'
ORDER BY pg_relation_size(indexrelid) DESC;
```

6. Tiga Jenis Scan — Kapan Planner Memilih Masing-Masing

Dari eksperimen Task 1–4, kamu sudah melihat ketiga jenis ini muncul. Ini penjelasan sistematis:

Scan Type	Kapan dipilih	Karakteristik
Seq Scan	Tabel kecil, atau query return >20-30% baris (low selectivity)	Bandwidthnya baris berurutan. Sequential I/O — cepat untuk data besar
Index Scan	Query return sedikit baris (high selectivity), biasanya <5%	Travel B-Tree + random fetch ke heap. Sangat cepat untuk lookup
Bitmap Index Scan	Query return baris medium (~5-20%), atau banyak baris	Bandwidth di RAM dulu, lalu batch-fetch heap. Kompromi antara
Index Only Scan	Semua kolom yang dibutuhkan ada di index (covered index)	Time index fetch heap sama sekali — tercepat dari semua jenis scan.

Kalimat kunci: Index Scan untuk yang sangat sedikit. Bitmap untuk yang medium — 'terlalu gemuk untuk Index Scan, terlalu berharga untuk Seq Scan.' Seq Scan untuk yang hampir semua.

7. Query Planner — Cara Kerjanya

Planner adalah optimizer yang berpikir sebelum mengeksekusi query. Dia tidak asal pilih index yang tersedia — dia estimasi cost setiap candidate plan dan pilih yang termurah.

Tiga input utama planner:

- **Statistik tabel (pg_statistic)** — disimpan oleh ANALYZE: berapa total baris, berapa nilai unik per kolom, distribusi nilai. Tanpa ini, planner 'buta' dan bisa salah pilih plan.
- **Index yang tersedia** — planner tahu semua index, tapi tetap hitung apakah memakai index lebih murah dari tidak memakai.
- **Cost model** — Sequential read 1 page = cost 1.0. Random read = cost 4.0. CPU per row = cost kecil. Planner jumlahkan semua untuk tiap candidate plan.

Contoh nyata dari Task 2: planner tidak pakai composite index yang baru dibuat karena query pakai LIMIT 10. Planner menghitung bahwa scan idx_created_at backward hanya perlu membaca 91 baris sebelum LIMIT terpenuhi — lebih murah dari composite index.

ANALYZE wajib setelah bulk INSERT besar. Di production, AUTOVACUUM akan trigger ANALYZE otomatis saat data berubah signifikan — tapi setelah load data manual tetap worth it untuk trigger manual.

8. Membaca EXPLAIN ANALYZE

Kolom penting yang harus dilihat:

Field	Arti	Yang harus diperhatikan
cost=X..Y	Estimasi planner (bukan ms)	X = startup cost, Y = total cost
actual time=A..B	Waktu eksekusi nyata dalam ms	B adalah total waktu aktual
rows=N	Estimasi baris yang dikembalikan	Bandingkan dengan actual rows — jika jauh beda, ANALYZE
Rows Removed by Filter	Baris dibaca tapi dibuang	Angka besar = Seq Scan tidak efisien
Heap Blocks: exact=N	Berapa page di-fetch dari heap	Muncul di Bitmap Heap Scan

```
-- Paksa Seq Scan untuk perbandingan (jangan di production)
SET enable_indexscan = OFF;
SET enable_bitmapscan = OFF;
EXPLAIN ANALYZE SELECT ...;
```

```
SET enable_indexscan = ON;  
SET enable_bitmapscan = ON;
```

9. Hasil Eksperimen Nyata (200k Baris)

Query	Tanpa Index	Dengan Index	Speedup
WHERE user_id = 4242	13.233 ms (Seq Scan)	0.275 ms (Bitmap)	~48×
WHERE score > 0.95	37.654 ms (Seq Scan)	4.624 ms (Bitmap)	~8×
model+status+ORDER BY LIMIT 10	9.680 ms (forced)	0.181 ms (Index Scan)	~53×

Catatan: score index menghasilkan Bitmap (bukan Index Scan biasa) karena 9.885 baris dikembalikan (~5% tabel) — terlalu banyak untuk Index Scan, terlalu sedikit untuk Seq Scan.

10. Key Takeaways

- Index bukan gratis — setiap write memperbarui semua index yang terdampak. Jangan index semua kolom.
- High cardinality (banyak nilai unik) + tabel besar = kondisi ideal untuk index.
- Low cardinality (status, boolean) = partial index atau tidak perlu index sama sekali.
- Planner memilih berdasarkan estimasi cost, bukan asumsi 'index = selalu lebih cepat'.
- ANALYZE wajib setelah bulk insert — planner butuh statistik akurat untuk memilih plan yang benar.
- Composite index: urutan kolom penting. Equality filter duluan, range/sort column terakhir.
- Partial index: sangat hemat space dan cepat untuk subset data yang sering di-query (352 kB vs 4416 kB).
- EXPLAIN ANALYZE adalah satu-satunya cara untuk tahu apakah index benar-benar dipakai planner.